

Plano Xclipse Open Project — XO940

Abordagem 5 dois meios e plano técnico baseado em evidências

Autor: Manus AI

Revisão: 05 de setembro de 2026

Alvo: Samsung SM-S721B / Xclipse 940

Sumário

1	Sumário executivo	1
2	1. Escopo e regra de evidência	1
3	2. Estado confirmado no pacote	2
4	3. Abordagem 5 dois meios	2
	4.1 3.1 Cinco objetivos críticos atuais	2
	4.2 3.2 Cinco descobertas rápidas	3
5	4. Ordem de execução e gates	3
6	5. Critérios de teste e encerramento	4
7	6. Entregáveis públicos do projeto	5
8	7. Referências	5
9	8. Atualização dos resultados obtidos	5
	9.1 8.1 Loader e caminho de produção	5
	9.2 8.2 Caminho SGPU e DRM	5
	9.3 8.3 Caminho real de compute	6
	9.4 8.4 Compiler, encoding e ISA	6
	9.5 8.5 Estado do conjunto anterior (histórico)	6
10	9. Prioridades atuais: Device Tree primeiro	7
	10.1 9.1 As cinco prioridades	7
	10.2 9.2 As cinco descobertas rápidas	7

1 Sumário executivo

Este plano atualiza o documento original do projeto **Xclipse Open Project** com o estado observado no pacote compartilhado e organiza a investigação pelo **abordagem 5 dois meios**: cinco descobertas essenciais para que o projeto possa avançar com segurança e cinco descobertas secundárias que podem aumentar a qualidade do driver, da documentação e da portabilidade.

O nome interno do projeto é **XO940**; o alvo de hardware é a **Xclipse 940**. O primeiro aparelho de laboratório é o **Samsung SM-S721B**, identificado nos artefatos como r12s, plataforma erd9945 e hardware s5e9945. O objetivo imediato não é declarar um driver funcional. É produzir uma especificação verificável do contrato entre aplicação, loader Android, ICD vendor, driver de usuário, DRM/UAPI, firmware e GPU. O pacote adicional SM-S721B.zip foi comparado por hash. Seu arquivo interno SM-S721B_16_0opensource.zip é byte-idêntico ao código-fonte Samsung já analisado; o pacote externo acrescenta Device Trees de variantes S7210, S721J, S721Q e S721U/S721W, além de pacotes específicos de builds. Essas variantes serão usadas para separar propriedades comuns da plataforma SGPU, diferenças de placa e diferenças de revisão.

> **Conclusão atual**: já existe um caminho SGPU observável por DRM para identificação, consultas, alocação GTT, mapeamento de CPU e mapeamento de VA. Ainda não existe evidência de submissão de comandos, dispatch compute, readback produzido pela GPU, ISA decodificada, backend de compilador ou driver Vulkan independente.

2 1. Escopo e regra de evidência

O projeto combina documentação de GPU, reverse engineering controlado, bring-up de DRM, investigação de loader Android, desenvolvimento futuro de ferramentas e, somente quando a evidência justificar, um driver Vulkan/Mesa experimental para Xclipse. “Turnip-like” descreve apenas a categoria de um driver Vulkan aberto para Xclipse; não implica compatibilidade ou reutilização de código do Turnip. O mesmo vale para RADV.

A regra central é separar **inicialização, observação, capability query, construção de pipeline, submissão, execução, sincronização e validação de resultado**. Um arquivo chamado probe, test, exec ou diagnostico não é um teste de capacidade por si só.

Artefato	Pode estabelecer	Não estabelece sozinho
Probe SGPU	Consultas DRM, memória e VA executadas	Que a GPU recebeu ou concluiu trabalho
Pipeline Vulkan	Que objetos de pipeline foram aceitos pelo caminho visível	Dispatch, submit, execução ou readback
Mapeamento de BO	Visibilidade e mapeamento de uma alocação	Acesso correto da GPU ou coerência de cache
Queue submit	Que uma API aceitou uma submissão	Que firmware executou o IB ou produziu resultado
Readback esperado	Uma operação fim a fim passou	Compatibilidade geral entre revisões e workloads

3 2. Estado confirmado no pacote

O pacote Quick Share foi preservado fora do Git e recebeu o hash SHA-256 cd73e2fa24ac083b9babf243b39772065b060abc9cd30fb476acabe74433e67f. A fonte Samsung foi inventariada sem redistribuir o código no repositório. O header `kernel/include/uapi/drm/sgpu_drm.h`, o módulo SGPU AMDGPU-derived, os Device Trees s5e9945, caminhos de IOMMU/DMA-BUF e referências de firmware foram localizados.

O relatório de dispositivo identifica o render node SGPU `/dev/dri/renderD128`, enquanto `/dev/dri/renderD129` pertence ao caminho `exynos-drm`. A consulta de hardware informa família 147 (MGFX), device ID 0x000073a0, revisão 0x02600200, GFX IP 10.0 com máscara de rings 0xf e COMPUTE IP 10.0 com máscara 0x7. O firmware observado inclui SGPU 2.23.0 e RTL 0x4ea15.

O probe SGPU criou um BO GTT de 64 KiB, realizou mapeamento pela CPU, tocou a memória, mapeou VA, desmapeou VA e fechou o objeto. O código declara explicitamente que **não submete nada à GPU**. O probe Vulkan expõe apenas `llvmpipe` no namespace Termux; a criação de pipeline não alcançou uma execução Samsung.

4 3. Abordagem 5 dois meios

O abordagem não significa escolher cinco testes arbitrários. Cada item precisa conter uma hipótese, uma forma de execução, um resultado mínimo, uma evidência bruta e uma regra de promoção. Os cinco primeiros itens bloqueiam decisões arquiteturais. Os cinco seguintes são valiosos, mas não devem atrasar a prova dos fundamentos.

4.1 3.1 Cinco objetivos críticos atuais

O objetivo de longo prazo é investigar um driver de espaço de usuário independente, com arquitetura comparável à de um driver Mesa/Vulkan, sem assumir que RADV possa ser copiado para a Xclipse. Cada contrato Device Tree, UAPI, memória, firmware, ring e Android precisa ser validado como específico.

N	Objetivo crítico	Evidência e investigação	Critério de saída
C1	Device Tree e plataforma	Fechar <code>sgpu@22200000</code> , regiões MMIO, IRQ, G3DCORE, clocks, DVFS, reset, IOMMU e variantes.	Mapa Device Tree → kernel SGPU → recursos reproduzido.
C2	Memória, IOMMU e VM	Reproduzir BO/GEM/DMA-BUF, heap, IOVA, VA, permissões, cache, flush e limpeza.	Ciclo de vida controlado com erro e cleanup observáveis.
C3	Job GFX controlado	Isolar context, seqno, PID, IB, scheduler e ring usando a evidência	Job próprio com fence correlacionada e buffer validado.

		de amdgpu_cs_ioctl e gfx_0.0.0.	
C4	Contrato UAPI/ABI	Separar kernel aberto, ABI vendor, namespaces, permissões e bibliotecas necessárias.	Tabela do cliente mínimo e dos bloqueios de reutilização.
C5	Primeiro workload independente	Tentar compute simples e reversível; OpenCL vendor é referência, não ABI pública automática.	Entrada conhecida → execução → saída conhecida, ou bloqueio localizado.

4.2 3.2 Cinco descobertas rápidas

N	Descoberta rápida	Como investigar	Utilidade
Q1	Comparar Device Trees SGPU	Comparar common.dtsi, EVT0 e includes por reg, IRQ, clocks, power e IOMMU.	Evita usar a variante errada.
Q2	Inventário runtime único	Consolidar DRM, sysfs, firmware, frequência, permissões, processos e FDs.	Elimina duplicação entre sessões.
Q3	Bibliotecas namespaces e	Mapear SONAME, NEEDED, BuildID, namespace e processo da cadeia vendor.	Localiza a primeira barreira externa.
Q4	Índice UAPI e símbolos	Classificar GEM, VM, CS, fences, firmware, GFX/COMPUTE e DVFS por função.	Acelera a correlação fonte/runtime.
Q5	Assinatura de job real	Seguir sched_job, context, seqno, PID, num_ibs, ring, VM flush e cleanup.	Cria causalidade sem fingir readback.

5 4. Ordem de execução e gates

A ordem é deliberadamente conservadora. Primeiro vêm identidade, loader e plataforma. Depois memória e VM. Em seguida vêm filas, sincronização e recuperação. Só então compute, ISA, compiler, Vulkan e driver.

Fase	Objetivo	Gate de saída
0	Governança, hash, inventário, licença e perguntas abertas	Manifesto, inventários e STATUS versionados
1	Plataforma, DRM, UAPI, DT, firmware e loader	Mapa reproduzível do caminho até a GPU visível
2	Observabilidade segura e probes read-only	Saídas brutas, comandos e limites preservados
3	Memória, VA, IOMMU, cache e faults	Alocação e mapeamento reproduzíveis
4	Queues, command processor, fences e reset	Hipótese de packet/ring apoiada por evidência
5	Compute mínimo	Submit, sincronização e readback esperado
6	ISA e compiler	Uma forma de instrução correlacionada com confiança
7–10	Vulkan, layers, driver e manutenção	Feature Vulkan real, layer observável e regressões versionadas

6 5. Critérios de teste e encerramento

Um teste real deve indicar aparelho, build, revisão, firmware, data, comando exato, saída bruta, interpretação e nível de confiança. O relatório deve declarar qual camada foi atingida. O projeto deve registrar também abordagens descartadas, como o dlopen direto no namespace Termux, sem transformar essa falha específica em uma afirmação universal.

Nenhum teste de submissão deve usar command buffers opacos, escrever registradores sem necessidade documentada, alterar permanentemente SELinux ou modificar /system e /vendor. Cada experimento precisa ter pré-condição, timeout, condição de abortar, recuperação e evidência de que o aparelho voltou ao estado esperado.

Rótulo	Uso	Exemplo Xclipse 940
Confirmado	Observado diretamente ou sustentado por fonte pública clara	renderD128 ligado ao SGPU; BO/VA probe concluído
Confirmado por fonte	Presente no kernel, documentação ou símbolo verificável	sgpu_drm.h, Device Tree e caminhos SGPU
Provável	Inferido de múltiplas evidências	Compatibilidade AMDGPU-derived além do subset observado

Hipótese	Possibilidade que exige experimento	Primeiro submit seguro em ring COMPUTE
Descartado	A abordagem específica falhou	Abrir ICD diretamente no namespace Termux

7 6. Entregáveis públicos do projeto

O repositório deve manter especificações por camada, inventários CSV, relatórios de experimentos, dados sanitizados do SM-S721B, classificações de probes e testes, e uma árvore que diferencie o que já existe do que está planejado. Código Samsung, firmware, bibliotecas vendor e dumps crus não são publicados automaticamente. A licença precisa ser avaliada por arquivo e a proveniência precisa acompanhar qualquer reutilização.

Este plano é um guia de trabalho, não um certificado de compatibilidade. O resultado bem-sucedido pode ser uma documentação suficiente para permitir decisões técnicas corretas, mesmo que alguma camada — loader, firmware, ISA, compiler ou reset — continue bloqueando um driver independente.

8 7. Referências

- [1] Arquivo Quick Share fornecido para o projeto Xclipse Open (nome interno XO940), com hash registrado em `data/source-archive-manifest.txt`.
- [2] `reports/initial-evidence.md`, relatório de evidências iniciais do pacote.
- [3] `source-analysis/kernel-source-review.md`, inventário curado do código Samsung.
- [4] `docs/00-project-scope.md`, escopo e regra de evidência do repositório.
- [5] `ROADMAP.md`, fases e critérios de saída do projeto.

9 8. Atualização dos resultados obtidos

O novo pacote `5doismeiosresultadosobtidos.zip` foi analisado em 05 de setembro de 2026. Seu SHA-256 é `59ac6221373572024c0967d642f9ead1f970c6791ff2428cb5c49ae73a5fe7fc`. O binário vendor foi mantido fora do repositório; somente metadados, caminhos e conclusões derivadas foram publicados.

9.1 8.1 Loader e caminho de produção

A coleta válida do SurfaceFlinger mostrou `/vendor/lib64/hw/vulkan.samsung.so` e `/system/lib64/libvulkan.so` mapeadas no processo, além de FDs apontando para `/dev/dri/renderD128`. O mount namespace do SurfaceFlinger era diferente do namespace Termux. Portanto, o resultado `llvmpipe` no Termux não invalida a presença do ICD Samsung no caminho de produção. A enumeração detalhada de `VkPhysicalDevice` nesse processo ainda não foi capturada.

9.2 8.2 Caminho SGPU e DRM

O pacote fecha o mapa `card0 → sgpu`, `card1 → exynos-drm`, `/dev/dri/renderD128 → SGPU` e `/dev/dri/renderD129 → Exynos/display`. A identidade `/sys/devices/platform/22200000.sgpu`, o compatível `samsung-sgpu,samsung-sgpu`, firmware SGPU 2.23.0, RTL `0x0004ea15` e frequências `devfreq` foram

correlacionados. A biblioteca `libdrm_sgpu.so` expõe, no inventário, interfaces de BO, VA, contexto, submit, fences e queries; isso mapeia capacidade, não prova de chamada executada.

9.3 8.3 Caminho real de compute

O processo correto do Photo Remaster foi identificado como `com.samsung.android.photoremasterservice:photoremasterservice`. Ele carregou `vulkan.samsung.so`, `libdrm_sgpu.so`, `libOpenCL.so` e `libSGPUOpenCL.so`, mantendo FDs em `renderD128` e `card0`. A API OpenCL inclui contexto, fila, kernel, `clEnqueueNDRangeKernel`, sincronização e `clEnqueueReadBuffer`. A atividade do processo correlacionou-se com `runtime_status=active` e variação de frequência SGPU. Ainda não há a cadeia comprovada B0 antes → submit → execução → fence/evento → readback → recuperação.

9.4 8.4 Compiler, encoding e ISA

A análise estática do ICD stripped encontrou vocabulário de SPIR-V, shader compiler, `GetOpcode`, `XlateOpcode`, `GetHwOpcode`, `SCEmitter`, `SCAsmEncoder`, `VOPC/VOP1/VOP2/VOP3/VOP3P`, `DPP8/DPP16` e mensagens de encoding. Uma disassembly AArch64 real foi obtida usando o endereço virtual correto da seção `.text, 0x143fb30`. A evidência é forte para infraestrutura interna de compiler/encoding, mas strings isoladas e referências que terminam em logging não são símbolos de função. `gfx10_4_GEN` também não prova que a Xclipse 940 seja GFX10.4. Ainda falta correlacionar shader controlado, binary gerado e ISA nativa da Xclipse 940 em runtime.

9.5 8.5 Estado do conjunto anterior (histórico)

Item	Estado	Conclusão correta
Loader Android	Parcialmente concluído	ICD e DRM observados no <code>SurfaceFlinger</code> ; enumeração Vulkan detalhada ainda aberta.
DRM/BO/VA/IOMMU	Fortalecido	Estrutura SGPU, firmware, <code>libdrm</code> e interfaces mapeadas; chamadas individuais ainda precisam de correlação.
Contexto/ring/IB/fence	Mapeado	APIs de submit e sync identificadas; submit do projeto não demonstrado.
Dispatch/readback	Caminho real identificado	Photo Remaster/OpenCL/SGPU correlacionados; kernel próprio com readback não provado.
Shader/binário/ISA	Evidência estática forte	Compiler/encoding interno sustentado; ISA nativa da Xclipse 940 em runtime ainda aberta.

O relatório operacional completo está em `reports/5-dois-meios-essenciais.md`, e a análise do pacote está em `reports/new-results-analysis.md`.

10 9. Prioridades atuais: Device Tree primeiro

O nome **Xclipse 940** designa o hardware. **XO940** é somente o nome interno do projeto Xclipse Open Project. A ordem de investigação foi substituída porque o caminho de plataforma precisa ser fechado antes de um submit ou dispatch.

10.1 9.1 As cinco prioridades

1. **Device Tree e plataforma:** fechar regiões de registradores, interrupções, power domains, clocks, DVFS, resets, IOMMU, DMA/coerência e diferenças de revisão.
2. **Memória e proteção:** fechar GEM/TTM, BO, DMA-BUF, heaps, IOMMU, VA, cache, buffers seguros, residency e page faults.
3. **Caminho vendor Android:** identificar processo, namespace, HAL/manifest, dependências, permissões DRM, SELinux e ABI.
4. **Firmware, filas e recuperação:** correlacionar firmware, GFX/COMPUTE/DMA, rings, IBs, doorbells, contextos, fences, syncobjs e reset com uma operação real.
5. **Compute e shader controlados:** capturar input conhecido, operação vendor, sincronização, output conhecido e, quando possível, binary/metadata do shader.

10.2 9.2 As cinco descobertas rápidas

1. Comparar os nós Device Tree `s5e9945-sgpu_common.dtsi` e `s5e9945-sgpu_evt0.dtsi`.
2. Consolidar nós DRM, sysfs, driver, firmware, frequência, permissões e processos com FDs em `renderD128`.
3. Inventariar BuildIDs, SONAMEs, NEEDED, arquitetura e dependências das bibliotecas vendor.
4. Indexar UAPI, Kconfig, Makefile e símbolos por GEM/VM/CS/firmware/reset/DVFS.
5. Catalogar strings e disassembly do ICD em níveis separados: presença textual, referência em código/log e comportamento confirmado.

A Device Tree é o contrato declarativo que informa ao kernel como o SGPU está conectado ao SoC. Ela não é o driver inteiro nem a árvore do repositório, mas é a base para interpretar endereços, interrupções, energia, clocks, DMA e IOMMU. O relatório completo está em `reports/5-dois-meios-essenciais.md`.